

Regime-Aware Portfolio Optimization Using Correlation Eigenstructure and Machine Learning-Based Market Regime Forecasting

Project Report

[https://github.com/k-means-kriss/
Regime-Aware-Portfolio-Optimization-Using-
Correlation-Eigenstructure-and-Machine-
Learning-Based-Mark](https://github.com/k-means-kriss/Regime-Aware-Portfolio-Optimization-Using-Correlation-Eigenstructure-and-Machine-Learning-Based-Mark)

September 8, 2026

Contents

1	Abstract	5
2	Introduction	5
3	Literature Review	5
3.1	Mean-Variance Portfolio Theory	5
3.2	Random Matrix Theory and Financial Correlations	6
3.3	Eigenstructure and Regime-Aware Portfolio Construction	6
3.4	Research Gap Addressed by This Project	6
4	Methodology: Mathematical Foundations	6
4.1	Problem Formulation and Definitions	6
4.2	Log Returns	7
4.3	Market Return and Realized Volatility	7
4.4	Cross-Sectional Dispersion	7
4.5	Rolling Correlation Matrix	8
4.6	Eigenvalue Decomposition	8

4.7	Dynamic Features	8
4.8	Regime Classification Mathematics	9
4.9	Portfolio Variance	9
4.10	Minimum-Variance Optimization	10
4.11	Eigenvector Exposure Constraint	10
4.12	Why the Constraint Changes Portfolio Weights	10
4.13	R^2 Comparison Used in the Project	11
4.14	Mathematical Validation Checks	11
5	Proposed Model / Framework	11
5.1	Overall Architecture	11
5.2	Experimental Setup and Specifications	12
6	Main Model Implementation	12
6.1	Shared Preprocessing	12
6.1.1	Imports and final universe	13
6.1.2	Log-return construction	13
6.1.3	Rolling eigenvalue decomposition	14
6.1.4	Market features and merge	14
6.2	Nowcast Model	15
6.2.1	Nowcast VIX Model	15
6.2.2	Nowcast regime classification and feature engineering	16
6.3	Forecast Model	16
6.3.1	Forecast VIX model	17
6.3.2	Forecast regime classification and feature engineering	18
6.4	Shared Feature Selection	18
6.4.1	Feature-set selection	19
6.5	Shared Classification Model	19
6.5.1	Classifier imports	20
6.5.2	Chronological train/test construction	20
6.5.3	Hard-voting ensemble	21
6.5.4	Classification evaluation	21

6.5.5	Time-series cross-validation	22
6.6	Shared Portfolio Overlay	22
6.6.1	Portfolio optimization core	23
6.7	Latest-Feature Extraction	23
6.7.1	Nowcast latest-feature extraction	24
6.7.2	Forecast latest-feature extraction	25
6.8	Regime-Driven Portfolio Decision	25
6.8.1	Nowcast regime-to-portfolio mapping	26
6.8.2	Forecast regime-to-portfolio mapping	27
7	Results	27
7.1	Classification Results	27
7.2	Portfolio Exposure and Variance Results	28
7.3	September 3 Nowcast Versus Forecast Result	29
7.4	September 3 Portfolio Weights	29
7.5	Similarity of the Two September 3 Portfolios	30
8	Benchmarks	30
8.1	Single Eigenvalue-Ratio Threshold Baseline	30
8.2	Unconstrained Minimum-Variance Portfolio	30
9	Experiments	31
9.1	Feature-Set Size Experiment	31
9.2	Nowcast Versus Forecast Experiment	31
9.3	Eigenvector Constraint Experiment	31
10	Ablation and Failure-Mode Analysis	32
10.1	VIX Label Leakage	32
10.2	Universe-Index Mismatch	32
10.3	Removal of Direct VIX From the Classifier	32
10.4	Feature-Set Ablation	32
11	Discussion	32
11.1	What the Model Actually Learns	32

11.2 Why the Portfolio Overlay Is Economically Meaningful	33
11.3 Interpretation of the September 3 Result	33
11.4 Backtesting Limitation	33
12 Future Improvements	33
13 Conclusion	34
14 Project Repository and Reproducibility	35
15 References	35

1 Abstract

This project develops a regime-aware portfolio construction framework that connects rolling correlation eigenstructure, machine-learning regime classification, and constrained minimum-variance portfolio optimization. The central problem is that diversification can weaken during market stress because asset correlations increase and a common market component can dominate the cross-sectional covariance structure.

Two regime classifiers are implemented. The first is a *nowcast* that classifies the regime observed on the current day. The second is a *forecast* that uses information observed on day t to classify the regime on day $t + 1$. Both models use a hard-voting ensemble of Random Forest, XGBoost, and LightGBM. The final compact feature set contains eight variables built from volatility, cross-sectional dispersion, and correlation-matrix eigenstructure.

The portfolio layer solves a long-only, fully invested minimum-variance problem. When the predicted regime is transitioning or stressed, the optimizer additionally caps the portfolio projection onto the leading eigenvector of the rolling correlation matrix. The supplied project results report approximately 90% cross-validated classification accuracy for both the nowcast and forecast tasks. A walk-forward backtest was attempted, but threshold-calibration sensitivity prevented a robust comparative performance claim.

2 Introduction

Modern portfolio theory relies on diversification: combining assets whose returns are not perfectly correlated can reduce aggregate portfolio variance. However, the benefit is state-dependent. During stress periods, correlations can increase and assets that were previously useful for diversification can become exposed to the same dominant market factor.

The project therefore investigates a pipeline in which the changing dependence structure of a small equity universe is measured using a rolling correlation matrix. Its eigenvalues summarize the strength of common directions, while its leading eigenvector provides a direction in asset space associated with the dominant co-movement pattern.

The project extends the eigenstructure mechanism discussed by Girnyk by replacing a single-signal regime rule with a multi-feature machine-learning classifier and by constructing both a same-day nowcast and a genuine next-day forecast. The supplied project document explicitly characterizes this work as inspired by Girnyk rather than as a replication study.

3 Literature Review

3.1 Mean-Variance Portfolio Theory

The portfolio optimization component is grounded in the mean-variance framework introduced by Markowitz. The original portfolio-selection problem formalized the trade-off between portfolio

risk and expected return and established variance as a central portfolio-risk measure. The present project uses the minimum-variance part of this framework rather than optimizing expected return.

3.2 Random Matrix Theory and Financial Correlations

Random Matrix Theory has been applied to financial correlation matrices to distinguish collective structure from noise. Large eigenvalues that depart from the random-matrix bulk can represent meaningful common components. The leading eigenvalue is therefore useful as a compact measure of the dominance of a common market direction.

Laloux et al. and Plerou et al. established influential applications of random-matrix methods to empirical stock-return correlation matrices. These studies provide the conceptual basis for examining eigenvalues and eigenvectors rather than treating the correlation matrix as an undifferentiated collection of pairwise coefficients.

3.3 Eigenstructure and Regime-Aware Portfolio Construction

Girnyk’s work on Nordic stock markets is the direct inspiration for this project. The source study examines time-varying correlation eigenstructure and reports that stress periods are associated with a rising leading eigenvalue and counter-cyclical behavior in the second eigenvalue. It also proposes limiting portfolio exposure to dominant eigenmodes during stress.

The present project adopts that core mechanism but changes the regime-detection layer. Instead of relying only on an eigenvalue-ratio threshold, it uses engineered features and an ensemble classifier. It additionally creates a one-step-ahead forecast model.

3.4 Research Gap Addressed by This Project

The project combines four elements into one executable pipeline:

rolling correlation eigenstructure $\rightarrow$ multi-feature regime classification $\rightarrow$ regime-conditioned exposure control

The resulting framework therefore treats eigenstructure not only as a descriptive statistic but as an input to an explicit portfolio decision.

4 Methodology: Mathematical Foundations

4.1 Problem Formulation and Definitions

Let $N = 6$ denote the number of stocks and let $P_{i,t}$ be the closing price of stock i at day t . The final universe used by the supplied notebooks is

$$\{\text{AAPL}, \text{JPM}, \text{JNJ}, \text{XOM}, \text{WMT}, \text{CAT}\}.$$

Let $r_{i,t}$ denote the log return, C_t the rolling correlation matrix, Σ_t the covariance matrix, $v_1(t)$ the leading eigenvector, and w_t the portfolio-weight vector.

The complete decision problem can be written as

$$\boxed{\text{prices} \rightarrow \text{returns} \rightarrow C_t \rightarrow (\lambda_1, \lambda_2, \lambda_3, v_1) \rightarrow \text{features} \rightarrow \widehat{\text{regime}} \rightarrow w_t}$$

where the final step is a constrained optimization problem.

4.2 Log Returns

The project uses

$$r_{i,t} = \ln \left(\frac{P_{i,t}}{P_{i,t-1}} \right).$$

For a sequence of prices,

$$\sum_{k=t+1}^{t+h} r_{i,k} = \ln \left(\frac{P_{i,t+h}}{P_{i,t}} \right),$$

which explains the usefulness of log returns for rolling calculations.

4.3 Market Return and Realized Volatility

The cross-sectional market return is

$$\bar{r}_t = \frac{1}{N} \sum_{i=1}^N r_{i,t}.$$

The project computes annualized 20-day realized volatility as

$$\sigma_{20,t} = \text{Std}(\bar{r}_{t-19:t})\sqrt{252}.$$

The factor $\sqrt{252}$ annualizes a daily standard deviation under the usual independence/scaling convention.

4.4 Cross-Sectional Dispersion

Cross-sectional dispersion is

$$D_t = \text{Std}_i(r_{i,t}).$$

This captures how differently the assets are behaving on the same trading day.

4.5 Rolling Correlation Matrix

For a 60-day window,

$$C_t = \text{Corr}(r_{t-59}, \dots, r_t).$$

The matrix is symmetric and positive semidefinite, so its eigenvalues are real and non-negative up to numerical precision.

4.6 Eigenvalue Decomposition

The spectral decomposition is

$$C_t = V_t \Lambda_t V_t^\top,$$

where

$$\Lambda_t = \text{diag}(\lambda_{1,t}, \dots, \lambda_{N,t}), \quad \lambda_{1,t} \geq \lambda_{2,t} \geq \dots \geq \lambda_{N,t}.$$

The project retains the first three eigenvalues and defines

$$R_t = \frac{\lambda_{1,t}}{\lambda_{2,t}}.$$

A high R_t indicates that the first common direction is strong relative to the second eigenmode.

4.7 Dynamic Features

For a generic signal x_t , the five-day change is

$$\Delta_5 x_t = x_t - x_{t-5}.$$

The project also calculates ten-day slopes using a linear fit over each rolling window. A rolling z-score is

$$z_t = \frac{x_t - \mu_{60,t}}{\sigma_{60,t}},$$

where $\mu_{60,t}$ and $\sigma_{60,t}$ are the trailing 60-day mean and standard deviation.

Other engineered quantities include:

$$G_t = \lambda_{2,t} - \lambda_{3,t},$$

$$\text{VolOfVol}_{10,t} = \text{Std}(\sigma_{20,t-9:t}),$$

rolling skewness, rolling volatility-dispersion correlation, return autocorrelation, drawdown, and downside volatility.

4.8 Regime Classification Mathematics

The stress proxy $\hat{V}_t$ is divided at empirical quantiles

$$Q_{33} = \text{Quantile}_{0.33}(\hat{V}), \quad Q_{66} = \text{Quantile}_{0.66}(\hat{V}).$$

The regime is

$$\text{Regime}_t = \begin{cases} \text{calm}, & \hat{V}_t \leq Q_{33}, \\ \text{transitioning}, & Q_{33} < \hat{V}_t \leq Q_{66}, \\ \text{stressed}, & \hat{V}_t > Q_{66}. \end{cases}$$

For the forecast model, the historical target is shifted so that features at t are mapped to a target associated with $t + 1$.

4.9 Portfolio Variance

For weights w and covariance matrix Σ ,

$$\text{Var}(R_p) = w^\top \Sigma w.$$

This follows from

$$R_p = w^\top R$$

and

$$\text{Var}(R_p) = \text{Var}(w^\top R) = w^\top \text{Cov}(R) w = w^\top \Sigma w.$$

4.10 Minimum-Variance Optimization

The baseline problem is

$$\begin{aligned} \min_w \quad & w^\top \Sigma w \\ \text{s.t.} \quad & \mathbf{1}^\top w = 1, \\ & w_i \geq 0, \quad i = 1, \dots, N. \end{aligned}$$

The equality constraint makes the portfolio fully invested. The inequality constraints make it long-only.

4.11 Eigenvector Exposure Constraint

Let v_1 be the eigenvector associated with λ_1 . The portfolio's exposure to this dominant direction is

$$E(w) = w^\top v_1.$$

During transitioning or stressed regimes, the project imposes

$$\boxed{|w^\top v_1| \leq 0.15}.$$

Equivalently,

$$-0.15 \leq w^\top v_1 \leq 0.15.$$

Because $w^\top v_1$ is linear in w , the additional constraint preserves convexity. The optimizer is therefore still solving a convex quadratic program.

4.12 Why the Constraint Changes Portfolio Weights

Suppose the unconstrained solution has

$$|w^\top v_1| > 0.15.$$

Then it is infeasible under the regime-aware problem. The optimizer must move to another point in the feasible region. It can do this by decreasing weights on stocks whose components of v_1 contribute strongly to the exposure and increasing weights on stocks that allow the portfolio to maintain

$$\mathbf{1}^\top w = 1$$

while satisfying the exposure bound.

Because the constrained feasible set is a subset of the unconstrained feasible set,

$$\mathcal{F}_{\text{constrained}} \subseteq \mathcal{F}_{\text{baseline}},$$

the optimal constrained variance cannot be lower than the unconstrained minimum:

$$V_{\text{constrained}} \geq V_{\text{baseline}}.$$

This is exactly the safety-versus-unconstrained-risk trade-off observed in the project output.

4.13 R^2 Comparison Used in the Project

For two vectors y and $\hat{y}$, the coefficient of determination is

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

The final comparison uses this statistic to compare the six portfolio weights produced by the nowcast and forecast pipelines for September 3 on the basis of September 2 features. This is a measure of similarity between two weight vectors; it is not a portfolio return metric.

4.14 Mathematical Validation Checks

Three theoretical checks are central to the project:

1. the relationship between λ_1 and λ_2 ;
2. the relationship between the leading eigenvector and the empirical market return;
3. comparison of the single eigenvalue-ratio threshold rule with the multi-feature classifier.

The supplied report records a Pearson correlation of approximately -0.62 between the first and second eigenvalues and an eigenportfolio regression with approximately $R^2 = 0.994$ and $\beta \approx 1.04$.

5 Proposed Model / Framework

5.1 Overall Architecture

The proposed framework has two branches after the common feature-engineering stage.

1. **Nowcast branch:** current-day engineered features $\rightarrow$ current-day stress proxy $\rightarrow$ current-day regime.
2. **Forecast branch:** current-day engineered features $\rightarrow$ next-day stress proxy $\rightarrow$ next-day regime.

Both branches then use the same classifier architecture and the same portfolio overlay.

5.2 Experimental Setup and Specifications

- Universe: AAPL, JPM, JNJ, XOM, WMT, CAT.
- Price history: approximately ten years for the main training dataset.
- Rolling correlation window: 60 trading days.
- Realized volatility window: 20 trading days.
- Primary classifier feature set: 8 variables.
- Training block: first 1,800 observations.
- Remaining observations: held-out test period.
- Classifier: hard voting over Random Forest, XGBoost, and LightGBM.
- Time-series cross-validation: 5 folds.
- Portfolio: long-only, fully invested.
- Eigenvector exposure cap: 0.15 when the regime is transitioning or stressed.

6 Main Model Implementation

This section shows the actual implementation from the supplied notebooks. The code is presented as full-cell images rather than fragmented listings. Each image contains the complete code block at once, with line numbers, so that a code cell is never visually split across pages.

6.1 Shared Preprocessing

The following code is common to both models. It imports the required packages and sets the effective final universe. The notebook initially defines a technology-only universe but then overwrites it with the six-stock universe shown below; the latter is therefore the effective universe.

6.1.1 Imports and final universe

The first block loads the numerical, data-acquisition, visualization, and later machine-learning/optimization dependencies. The second universe assignment is the one that remains active in the notebook.

Shared imports and final universe

```
1  import yfinance as yf
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import seaborn as sns
6
7  universe = [
8      "NVDA", # NVIDIA
9      "AAPL", # Apple
10     "MSFT", # Microsoft
11     "GOOGL", # Alphabet
12     "AMZN", # Amazon
13     "META"  # Meta
14 ]
15
16 universe = ['AAPL', 'JPM', 'JNJ', 'XOM', 'WMT', 'CAT']
```

Figure 1: Complete shared import and universe code.

6.1.2 Log-return construction

The function downloads ten years of closing-price history and converts each stock price series into log returns. The resulting DataFrame is the input to every subsequent rolling statistic.

Log-return construction

```
1  def log_returns_data(universe):
2
3      data=pd.DataFrame()
4
5      for i in universe:
6          ticker=yf.Ticker(i)
7          df=ticker.history(period='10y')
8          name=f"log_returns_{i}"
9          data[name]=np.log(df['Close']/df['Close'].shift(1))
10
11     return data.dropna()
12
13 data=log_returns_data(universe)
14 data.tail()
15
16 stock_cols=data.columns
```

Figure 2: Complete log-return construction code.

6.1.3 Rolling eigenvalue decomposition

For each day after the first 60 observations, the code forms a 60-day correlation matrix, computes its eigenvalues, sorts them from largest to smallest, and retains the first three. It then constructs the leading/second eigenvalue ratio.

```
1 def compute_rolling_eigenvalues(data, window=60):
2     leading_eigenvalues = []
3     second_eigenvalues = []
4     third_eigenvalues = [] # <-- new
5     dates = []
6
7     for i in range(window, len(data)):
8         window_data = data.iloc[i-window:i]
9         corr_matrix = window_data.corr()
10
11         eigenvalues = np.linalg.eigvalsh(corr_matrix)
12         eigenvalues_sorted = eigenvalues[::-1]
13
14         leading_eigenvalues.append(eigenvalues_sorted[0])
15         second_eigenvalues.append(eigenvalues_sorted[1])
16         third_eigenvalues.append(eigenvalues_sorted[2]) # <-- new
17         dates.append(data.index[i])
18
19     eigen_df = pd.DataFrame({
20         'leading_eigenvalue': leading_eigenvalues,
21         'second_eigenvalue': second_eigenvalues,
22         'third_eigenvalue': third_eigenvalues # <-- new
23     }, index=dates)
24
25     eigen_df['eigenvalue_ratio'] = eigen_df['leading_eigenvalue'] / eigen_df['second_eigenvalue']
26     eigen_df.index = eigen_df.index.tz_localize(None)
27
28     return eigen_df
29
30
31 eigen_df=compute_rolling_eigenvalues(data,60)
32
```

Figure 3: Complete 60-day rolling eigenvalue code.

6.1.4 Market features and merge

The common preprocessing layer computes 20-day realized volatility, cross-sectional dispersion, and the project's in-universe VIX-like volatility proxy. The eigenvalue features are then merged with these market features by date. In the forecast branch, the next-day VIX target is also created at this stage.

```
Market features, VIX proxy, and merge

1 def add_vol_desperssion_vix(n):
2     n['market_realized_vol_20d']=n.mean(axis=1).rolling(20).std()*np.sqrt(252)
3     n['market_cross_sectional_dispersion']=n.std(axis=1)
4     n['VIX']=n.mean(axis=1).rolling(20).std()
5
6     return n
7
8 ndata=add_vol_desperssion_vix(data.copy())
9
10 def final(e,n):
11     final_data=n.merge(e,how='inner',right_index=True,left_index=True)
12     return final_data
13
14 # Ensure ndata's index is timezone-naive before merging
15 ndata.index = ndata.index.tz_localize(None)
16 final_data=final(eigen_df.copy(),ndata.copy())
17
18 final_data['vix_-1']=final_data['VIX'].shift(-1)
19 final_data.dropna(inplace=True)
```

Figure 4: Market feature, merge, and forecast-target preparation code.

6.2 Nowcast Model

6.2.1 Nowcast VIX Model

This is the first component unique to the nowcast pipeline. Its target is the same-day VIX proxy, so the resulting `pred_vix` is a same-day stress estimate. The code then removes the real VIX columns before the classifier is trained.

```
1  def vix_model(data):
2      data.drop((data.select_dtypes(include='object').columns),axis=1,inplace=True)
3
4      from sklearn.model_selection import train_test_split as tts , GridSearchCV as grid
5      ,RandomizedSearchCV as rand
6      from sklearn.linear_model import LinearRegression
7      from sklearn.ensemble import RandomForestRegressor,VotingRegressor
8      from xgboost import XGBRegressor
9      from sklearn.neighbors import KNeighborsRegressor
10     from sklearn.naive_bayes import GaussianNB
11     from sklearn.metrics import accuracy_score,classification_report,r2_score
12     from sklearn.tree import DecisionTreeRegressor
13     from lightgbm import LGBMRegressor
14     train=data[:1800]
15     test=data[1800:]
16
17     x=train.drop(['VIX','vix_-1'],axis=1)
18     y=train['VIX']
19
20     xtrain,xtest,ytrain,ytest=tts(x,y,shuffle=False,test_size=0.2)
21
22     model=VotingRegressor([
23         ('rfr',RandomForestRegressor(random_state=42)),
24         ('xgb',XGBRegressor(random_state=42)),
25         ('lr',LinearRegression()),
26     ])
27     model.fit(xtrain,ytrain)
28
29     pred=model.predict(data.drop(['VIX','vix_-1'],axis=1))
30
31     return pred
32
33 col=vix_model(final_data)
34 col
35
36 final_data['pred_vix']=col
37 final_data.drop(['VIX','vix_-1'],axis=1,inplace=True)
```

Figure 5: Complete nowcast VIX model and proxy-output code.

6.2.2 Nowcast regime classification and feature engineering

This is the second unique nowcast component. The first part determines the regime class from the predicted current-day stress signal. The remainder creates the dynamic features needed by the final classifier.

```
1 lower=final_data['pred_vix'].quantile(0.33)
2 upper=final_data['pred_vix'].quantile(0.66)
3
4 l=[]
5 for i in final_data['pred_vix']:
6     if i<=lower:
7         l.append('calm')
8     elif lower<i<=upper:
9         l.append('transitioning')
10    else:
11        l.append('stressed')
12    final_data['regime']=l
13
14 import numpy as np
15 import pandas as pd
16
17 # --- Third eigenvalue (requires modifying compute_rolling_eigenvalues to also store it) ---
18 # If not already done, add this inside that function's loop:
19 # third_eigenvalues.append(eigenvalues_sorted[2])
20 # and this to eigen_df construction:
21 # eigen_df['third_eigenvalue'] = third_eigenvalues
22 # Then it flows into final_data automatically via the merge you already did.
23
24
25 # --- Rolling slope helper ---
26 def rolling_slope(series, window=10):
27     return series.rolling(window).apply(
28         lambda x: np.polyfit(range(len(x)), x, 1)[0], raw=True
29     )
30
31 # Rate-of-change features – capture the dynamics of regime shift, not just the level
32 final_data['eigenvalue_ratio_diff5'] = final_data['eigenvalue_ratio'].diff(5)
33 final_data['vol_diff5'] = final_data['market_realized_vol_20d'].diff(5)
34 final_data['dispersion_diff5'] = final_data['market_cross_sectional_dispersion'].diff(5)
35
36 # diff() introduces NaN for the first 5 rows (no prior value to compare against)
37 # --- New features ---
38 final_data['eigenvalue_ratio_slope10'] = rolling_slope(final_data['eigenvalue_ratio'], 10)
39 final_data['vol_slope10'] = rolling_slope(final_data['market_realized_vol_20d'], 10)
40 final_data['eigenvalue_gap_2_3'] = final_data['second_eigenvalue'] - final_data['third_eigenvalue']
41 final_data['vol_of_vol_10d'] = final_data['market_realized_vol_20d'].rolling(10).std()
42 final_data['market_return'] = final_data.select_dtypes(include=np.number).mean(axis=1)
43 final_data['market_return_skew_20d'] = final_data['market_return'].rolling(20).skew()
44 # 3. Rolling correlation between volatility and dispersion – divergence can signal transition
45 final_data['vol_dispersion_corr_20d'] = final_data['market_realized_vol_20d'].rolling(20).corr(
46     final_data['market_cross_sectional_dispersion']
47 )
48
49 # 4. Return autocorrelation – crashes/panics often show negative autocorrelation (sharp reversals)
50 final_data['return_autocorr_20d'] = final_data['market_return'].rolling(20).apply(
51     lambda x: pd.Series(x).autocorr(lag=1), raw=False
52 )
53
54 # 5. Rolling z-score of today's volatility vs its own recent history – "how unusual is today
55 specifically"
56 final_data['vol_zscore_60d'] = (
57     final_data['market_realized_vol_20d'] - final_data['market_realized_vol_20d'].rolling(60).mean()
58 ) / final_data['market_realized_vol_20d'].rolling(60).std()
59
60 # 6. Dispersion z-score, same idea, for cross-sectional dispersion
61 final_data['dispersion_zscore_60d'] = (
62     final_data['market_cross_sectional_dispersion'] -
63     final_data['market_cross_sectional_dispersion'].rolling(60).mean()
64 ) / final_data['market_cross_sectional_dispersion'].rolling(60).std()
65
66 cum_return = final_data['market_return'].cumsum()
67 rolling_max = cum_return.rolling(20).max()
68 final_data['drawdown_20d'] = cum_return - rolling_max
69
70 downside_returns = final_data['market_return'].clip(upper=0)
71 final_data['downside_vol_20d'] = downside_returns.rolling(20).std()
```

Figure 6: Complete nowcast regime-class and dynamic-feature code.

6.3 Forecast Model

6.3.1 Forecast VIX model

This is the first forecast-specific component. Unlike the nowcast VIX model, the regression target is the shifted next-day VIX proxy. The future value is used only as the historical training target, not as an input feature.

```
1 final_data['vix_-1']=final_data['VIX'].shift(-1)
2 final_data.dropna(inplace=True)
3
4 def vix_model(data):
5     data.drop((data.select_dtypes(include='object').columns),axis=1,inplace=True)
6
7     from sklearn.model_selection import train_test_split as tts , GridSearchCV as grid
8     ,RandomizedSearchCV as rand
9     from sklearn.linear_model import LinearRegression
10    from sklearn.ensemble import RandomForestRegressor,VotingRegressor
11    from xgboost import XGBRegressor
12    from sklearn.neighbors import KNeighborsRegressor
13    from sklearn.naive_bayes import GaussianNB
14    from sklearn.metrics import accuracy_score,classification_report,r2_score
15    from sklearn.tree import DecisionTreeRegressor
16    from lightgbm import LGBMRegressor
17    train=data[:1800]
18    test=data[1800:]
19
20    x=train.drop(['VIX','vix_-1'],axis=1)
21    y=train['vix_-1']
22
23    xtrain,xtest,ytrain,ytest=tts(x,y,shuffle=False,test_size=0.2)
24
25    model=VotingRegressor([
26        ('rfr',RandomForestRegressor(random_state=42)),
27        ('xgb',XGBRegressor(random_state=42)),
28        ('lr',LinearRegression()),
29    ])
30    model.fit(xtrain,ytrain)
31
32    pred=model.predict(data.drop(['VIX','vix_-1'],axis=1))
33
34    return pred
35
36 col=vix_model(final_data)
37 col
38
39 final_data['pred_vix']=col
```

Figure 7: Complete forecast VIX model and next-day target code.

6.3.2 Forecast regime classification and feature engineering

The forecast branch converts its predicted next-day stress proxy into the same three regime classes and constructs the same dynamic feature family. The thresholding rule is structurally the same as the nowcast rule, but the input stress signal corresponds to the forecast target. This creates a genuine one-step-ahead regime label for model training.

```
1 lower=final_data['pred_vix'].quantile(0.33)
2 upper=final_data['pred_vix'].quantile(0.66)
3
4 l=[]
5 for i in final_data['pred_vix']:
6     if i<=lower:
7         l.append('calm')
8     elif lower<=i<=upper:
9         l.append('transitioning')
10    else:
11        l.append('stressed')
12    final_data['regime']=l
13
14    import numpy as np
15    import pandas as pd
16
17    # --- Third eigenvalue (requires modifying compute_rolling_eigenvalues to also store it) ---
18    # If not already done, add this inside that function's loop:
19    # third_eigenvalues.append(eigenvalues_sorted[2])
20    # and this to eigen_df construction:
21    # eigen_df['third_eigenvalue'] = third_eigenvalues
22    # Then it flows into final_data automatically via the merge you already did.
23
24
25    # --- Rolling slope helper ---
26    def rolling_slope(series, window=10):
27        return series.rolling(window).apply(
28            lambda x: np.polyfit(range(len(x)), x, 1)[0], raw=True
29        )
30
31    # Rate-of-change features – capture the dynamics of regime shift, not just the level
32    final_data['eigenvalue_ratio_diff5'] = final_data['eigenvalue_ratio'].diff(5)
33    final_data['vol_diff5'] = final_data['market_realized_vol_20d'].diff(5)
34    final_data['dispersion_diff5'] = final_data['market_cross_sectional_dispersion'].diff(5)
35
36    # diff() introduces NaN for the first 5 rows (no prior value to compare against)
37    # --- New features ---
38    final_data['eigenvalue_ratio_slope10'] = rolling_slope(final_data['eigenvalue_ratio'], 10)
39    final_data['vol_slope10'] = rolling_slope(final_data['market_realized_vol_20d'], 10)
40    final_data['eigenvalue_gap_2_3'] = final_data['second_eigenvalue'] - final_data['third_eigenvalue']
41    final_data['vol_of_vol_10d'] = final_data['market_realized_vol_20d'].rolling(10).std()
42    final_data['market_return'] = final_data.select_dtypes(include=np.number).mean(axis=1)
43    final_data['market_return_skew_20d'] = final_data['market_return'].rolling(20).skew()
44    # 3. Rolling correlation between volatility and dispersion – divergence can signal transition
45    final_data['vol_dispersion_corr_20d'] = final_data['market_realized_vol_20d'].rolling(20).corr(
46        final_data['market_cross_sectional_dispersion']
47    )
48
49    # 4. Return autocorrelation – crashes/panics often show negative autocorrelation (sharp reversals)
50    final_data['return_autocorr_20d'] = final_data['market_return'].rolling(20).apply(
51        lambda x: pd.Series(x).autocorr(lag=1), raw=False
52    )
53
54    # 5. Rolling z-score of today's volatility vs its own recent history – "how unusual is today
55    # specifically"
56    final_data['vol_zscore_60d'] = (
57        final_data['market_realized_vol_20d'] - final_data['market_realized_vol_20d'].rolling(60).mean()
58    ) / final_data['market_realized_vol_20d'].rolling(60).std()
59
60    # 6. Dispersion z-score, same idea, for cross-sectional dispersion
61    final_data['dispersion_zscore_60d'] = (
62        final_data['market_cross_sectional_dispersion'] -
63        final_data['market_cross_sectional_dispersion'].rolling(60).mean()
64    ) / final_data['market_cross_sectional_dispersion'].rolling(60).std()
65
66    cum_return = final_data['market_return'].cumsum()
67    rolling_max = cum_return.rolling(20).max()
68    final_data['drawdown_20d'] = cum_return - rolling_max
69
70    downside_returns = final_data['market_return'].clip(upper=0)
71    final_data['downside_vol_20d'] = downside_returns.rolling(20).std()
72
73    final_data = final_data.dropna()
```

Figure 8: Complete forecast regime-class and dynamic-feature code.

6.4 Shared Feature Selection

The notebooks test a larger engineered feature set and then overwrite it with the final compact eight-feature set. The final variables are:

1. market realized volatility (20-day);
2. market cross-sectional dispersion;

3. leading eigenvalue;
4. second eigenvalue;
5. eigenvalue ratio;
6. five-day eigenvalue-ratio change;
7. five-day volatility change;
8. five-day dispersion change.

6.4.1 Feature-set selection

The first list represents the larger candidate specification. The second assignment is the final compact feature set actually used by the classifier.

```
Feature-set comparison and final eight features  
1  pred=model.predict(xtest)  
2  round(accuracy_score(ytest,pred),2)  
3  
4  print(classification_report(ytest, model.predict(xtest)))
```

Figure 9: Candidate and final feature-selection code.

6.5 Shared Classification Model

The classifier architecture is identical after the nowcast and forecast labels have been constructed.

6.5.1 Classifier imports

The classifier imports include several candidate algorithms. The final voting ensemble uses Random Forest, XGBoost, and LightGBM.

```
Classifier imports
1  from sklearn.model_selection import train_test_split as tts , GridSearchCV as grid ,RandomizedSearchCV as rand
2  from sklearn.linear_model import LogisticRegression
3  from sklearn.ensemble import RandomForestClassifier,VotingClassifier
4  from xgboost import XGBClassifier
5  from sklearn.neighbors import KNeighborsClassifier
6  from sklearn.naive_bayes import GaussianNB
7  from sklearn.metrics import accuracy_score,classification_report
8  from lightgbm import LGBMClassifier
9  from sklearn.tree import DecisionTreeClassifier
```

Figure 10: Classifier import code.

6.5.2 Chronological train/test construction

The data are divided chronologically rather than randomly. This preserves the time ordering of the financial observations.

```
Chronological train/test split
1  train=final_data[:1800]
2  test=final_data[1800:]
3
4  x=train[feature_cols]
5  y=train['regime']
6
7  xtrain,xtest,ytrain,ytest=tts(x,y,shuffle=False,test_size=0.2)
```

Figure 11: Training/test split code.

6.5.3 Hard-voting ensemble

The final classifier is a hard-voting ensemble. Each constituent model produces a class prediction and the voting classifier chooses the majority class.

Hard-voting ensemble

```
1  model = VotingClassifier([
2      ('rfc', RandomForestClassifier(random_state=42)),
3      ('xgb', XGBClassifier(random_state=42)),
4      ('lgb', LGBMClassifier(random_state=42))
5  ], voting='hard')
6
7  model.fit(xtrain,ytrain)
```

Figure 12: Voting-classifier construction and fitting code.

6.5.4 Classification evaluation

The model is evaluated using the validation score, rounded accuracy, a per-class classification report, and accuracy on the held-out test period.

Classification evaluation

```
1  model.score(xtest,ytest)
2
3  pred=model.predict(xtest)
4  round(accuracy_score(ytest,pred),2)
5
6  print(classification_report(ytest, model.predict(xtest)))
7
8  x=test[feature_cols]
9  y=test['regime']
10
11 pred=model.predict(x)
12 print((accuracy_score(y,pred)))
```

Figure 13: Classification evaluation code.

6.5.5 Time-series cross-validation

Five expanding time-series folds are used as an additional robustness check. The mean and standard deviation across folds are reported.

```
Five-fold time-series cross-validation

1  from sklearn.model_selection import TimeSeriesSplit
2  from sklearn.metrics import accuracy_score
3  import numpy as np
4
5  X = final_data[feature_cols]
6  y = final_data['regime']
7
8  tscv = TimeSeriesSplit(n_splits=5)
9
10 scores = []
11
12 for fold, (train_idx, test_idx) in enumerate(tscv.split(X)):
13     X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
14     y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
15
16     model.fit(X_train, y_train)
17     pred = model.predict(X_test)
18     acc = accuracy_score(y_test, pred)
19     scores.append(acc)
20
21     print(f"Fold {fold+1}: accuracy = {acc:.3f}, train size = {len(X_train)}, test size = {len(X_test)}")
22
23 print()
24 print(f"Mean accuracy: {np.mean(scores):.3f}")
25 print(f"Std deviation: {np.std(scores):.3f}")
```

Figure 14: Five-fold time-series cross-validation code.

6.6 Shared Portfolio Overlay

The portfolio layer is shared by the two regime models. It first constructs the covariance matrix and leading eigenvector, then solves the unconstrained minimum variance problem and the eigenvector-constrained problem.

6.6.1 Portfolio optimization core

The code estimates the covariance matrix, computes the leading eigenvector using `np.linalg.eigh`, solves the long-only fully invested baseline, and constructs the exposure-constrained portfolio with $|w^\top v_1| \leq 0.15$. The comparison table also reports how individual stock weights change.

```
1 import numpy as np
2 import cvxpy as cp
3 import pandas as pd
4
5 stock_names = universe
6
7 n = len(stock_cols)
8
9 # --- 1. Covariance matrix (for portfolio variance) ---
10 cov_matrix = final_data[stock_cols].cov().values
11
12 # --- 2. Correlation matrix + leading eigenvector (for the regime-aware constraint) ---
13 recent_window = final_data[stock_cols].iloc[-60:]
14 corr_matrix = recent_window.corr()
15
16 eigenvalues, eigenvectors = np.linalg.eigh(corr_matrix)
17 leading_eigenvector = eigenvectors[:, -1] # eigh returns ascending order; last column = leading
18 v = leading_eigenvector
19
20 # --- 3. Baseline portfolio (no regime awareness) ---
21 w = cp.Variable(n)
22 portfolio_variance = cp.quad_form(w, cov_matrix)
23 constraints = [cp.sum(w) == 1, w >= 0]
24 problem = cp.Problem(cp.Minimize(portfolio_variance), constraints)
25 problem.solve()
26
27 baseline = w.value
28
29 # --- 4. Constrained portfolio (regime = stressed, cap exposure to leading eigenvector) ---
30 w3 = cp.Variable(n)
31 portfolio_variance3 = cp.quad_form(w3, cov_matrix)
32 exposure3 = w3 @ v
33
34 constraints3 = [
35     cp.sum(w3) == 1,
36     w3 >= 0,
37     cp.abs(exposure3) <= 0.15 # tunable threshold
38 ]
39 problem3 = cp.Problem(cp.Minimize(portfolio_variance3), constraints3)
40 problem3.solve()
41
42 constrained = w3.value
43
44 # --- 5. Comparison table ---
45 def classify_change(base, new):
46     diff = new - base
47     pct_of_base = diff / base if base != 0 else 0
48
49     if abs(pct_of_base) < 0.05:
50         return "~flat"
51     elif pct_of_base >= 0.5:
52         return "↑↑ (big jump)"
53     elif pct_of_base > 0:
54         return "↑"
55     elif pct_of_base <= -0.5:
56         return "↓↓ (big drop)"
57     else:
58         return "↓"
59
60 comparison = pd.DataFrame({
61     'Stock': stock_names,
62     'Baseline': np.round(baseline, 3),
63     'Constrained': np.round(constrained, 3),
64 })
65 comparison['Change'] = [classify_change(b, c) for b, c in zip(baseline, constrained)]
66
67 print("Exposure (baseline): ", baseline @ v)
68 print("Exposure (constrained):", constrained @ v)
69 print("Variance (baseline): ", problem.value)
70 print("Variance (constrained):", problem3.value)
71 print()
72 print(comparison.to_string(index=False))
```

Figure 15: Complete covariance, eigenvector, baseline, and constrained portfolio code.

6.7 Latest-Feature Extraction

The live/current-date part of the pipeline differs slightly between the nowcast and forecast notebooks, so both implementations are shown separately.

6.7.1 Nowcast latest-feature extraction

The nowcast pipeline downloads recent history, reconstructs the rolling features, and extracts the latest row used for same-day regime classification. Its data slice uses the most recent observations up to the available endpoint.

```
1 import yfinance as yf
2 import numpy as np
3 import pandas as pd
4
5 def latest_feature_extraction(tickers, window=60, vol_window=20):
6     stock_cols = [f'log_returns_{t}' for t in tickers]
7
8     # 1. Pull enough history to fill all rolling windows (60-day eigenvalue window + buffer)
9     prices = yf.download(tickers, period='2y')['Close'][:-1]
10    prices.columns = [f'log_returns_{t}' for t in prices.columns]
11
12    # 2. Log returns
13    data = np.log(prices / prices.shift(1)).dropna()
14
15    # 3. Market return, vol, dispersion
16    data['market_return'] = data[stock_cols].mean(axis=1)
17    data['market_realized_vol_20d'] = data['market_return'].rolling(vol_window).std() * np.sqrt(252)
18    data['market_cross_sectional_dispersion'] = data[stock_cols].std(axis=1)
19
20    # 4. Rolling eigenvalues (leading, second, third) + ratio
21    leading_eig, second_eig, third_eig, dates = [], [], [], []
22    for i in range(window, len(data)):
23        window_data = data[stock_cols].iloc[i-window:i]
24        corr_matrix = window_data.corr()
25        eigenvalues = np.linalg.eigvalsh(corr_matrix)
26        eigenvalues_sorted = eigenvalues[::-1]
27        leading_eig.append(eigenvalues_sorted[0])
28        second_eig.append(eigenvalues_sorted[1])
29        third_eig.append(eigenvalues_sorted[2])
30        dates.append(data.index[i])
31
32    eigen_df = pd.DataFrame({
33        'leading_eigenvalue': leading_eig,
34        'second_eigenvalue': second_eig,
35        'third_eigenvalue': third_eig
36    }, index=dates)
37    eigen_df['eigenvalue_ratio'] = eigen_df['leading_eigenvalue'] / eigen_df['second_eigenvalue']
38
39    data = data.merge(eigen_df, left_index=True, right_index=True, how='inner')
40
41    # 5. Diff / slope / gap / vol-of-vol / skew / dispersion-corr / autocorr / z-scores / drawdown
42    / downside vol
43    def rolling_slope(series, w=10):
44        return series.rolling(w).apply(lambda x: np.polyfit(range(len(x)), x, 1)[0], raw=True)
45
46    data['eigenvalue_ratio_diff5'] = data['eigenvalue_ratio'].diff(5)
47    data['vol_diff5'] = data['market_realized_vol_20d'].diff(5)
48    data['dispersion_diff5'] = data['market_cross_sectional_dispersion'].diff(5)
49    data['eigenvalue_ratio_slope10'] = rolling_slope(data['eigenvalue_ratio'], 10)
50    data['vol_slope10'] = rolling_slope(data['market_realized_vol_20d'], 10)
51    data['eigenvalue_gap_2_3'] = data['second_eigenvalue'] - data['third_eigenvalue']
52    data['vol_of_vol_10d'] = data['market_realized_vol_20d'].rolling(10).std()
53    data['market_return_skew_20d'] = data['market_return'].rolling(20).skew()
54    data['vol_dispersion_corr_20d'] = data['market_realized_vol_20d'].rolling(20).corr(data['market_cross_sectional_dispersion'])
55    data['return_autocorr_20d'] = data['market_return'].rolling(20).apply(lambda x:
56    pd.Series(x).autocorr(lags=1, raw=False)
57    data['vol_zscore_60d'] = (data['market_realized_vol_20d'] -
58    data['market_realized_vol_20d'].rolling(60).mean()) /
59    data['market_realized_vol_20d'].rolling(60).std()
60    data['dispersion_zscore_60d'] = (data['market_cross_sectional_dispersion'] -
61    data['market_cross_sectional_dispersion'].rolling(60).mean()) /
62    data['market_cross_sectional_dispersion'].rolling(60).std()
63
64    cum_return = data['market_return'].cumsum()
65    data['drawdown_20d'] = cum_return - cum_return.rolling(20).max()
66    data['downside_vol_20d'] = data['market_return'].clip(upper=0).rolling(20).std()
67
68    data = data.dropna()
69
70    # 6. Return only the latest available row
71    return data.iloc[-1:]
72
73 tickers = universe
74 latest_row = latest_feature_extraction(tickers)
75 print(latest_row)
76
77 latest_features=latest_row[feature_cols]
```

Figure 16: Complete nowcast latest-feature extraction code.

6.7.2 Forecast latest-feature extraction

The forecast pipeline performs the same reconstruction but retains the specific forecast-notebook implementation, including its handling of the latest downloaded history and VIX-like feature.

```
1 import yfinance as yf
2 import numpy as np
3 import pandas as pd
4
5 def latest_feature_extraction(tickers, window=60, vol_window=20):
6     stock_cols = [f'log_returns_{t}' for t in tickers]
7
8     # 1. Pull enough history to fill all rolling windows (60-day eigenvalue window + buffer)
9     prices = yf.download(tickers, period='2y')['Close'][:2]
10    prices.columns = [f'log_returns_{t}' for t in prices.columns]
11
12    # 2. Log returns
13    data = np.log(prices / prices.shift(1)).dropna()
14    data['VIX'] = data.mean(axis=1).rolling(20).std()
15
16    # 3. Market return, vol, dispersion
17    data['market_return'] = data[stock_cols].mean(axis=1)
18    data['market_realized_vol_20d'] = data['market_return'].rolling(vol_window).std() * np.sqrt(252)
19    data['market_cross_sectional_dispersion'] = data[stock_cols].std(axis=1)
20
21    # 4. Rolling eigenvalues (leading, second, third) + ratio
22    leading_eig, second_eig, third_eig, dates = [], [], [], []
23    for i in range(window, len(data)):
24        window_data = data[stock_cols].iloc[i-window:i]
25        corr_matrix = window_data.corr()
26        eigenvalues = np.linalg.eigvalsh(corr_matrix)
27        eigenvalues_sorted = eigenvalues[::-1]
28        leading_eig.append(eigenvalues_sorted[0])
29        second_eig.append(eigenvalues_sorted[1])
30        third_eig.append(eigenvalues_sorted[2])
31        dates.append(data.index[i])
32
33    eigen_df = pd.DataFrame({
34        'leading_eigenvalue': leading_eig,
35        'second_eigenvalue': second_eig,
36        'third_eigenvalue': third_eig
37    }, index=dates)
38    eigen_df['eigenvalue_ratio'] = eigen_df['leading_eigenvalue'] / eigen_df['second_eigenvalue']
39
40    data = data.merge(eigen_df, left_index=True, right_index=True, how='inner')
41
42    # 5. Diff / slope / gap / vol-of-vol / skew / dispersion-corr / autocorr / z-scores / drawdown
43    / downside vol
44    def rolling_slope(series, w=10):
45        return series.rolling(w).apply(lambda x: np.polyfit(range(len(x)), x, 1)[0], raw=True)
46
47    data['eigenvalue_ratio_diff5'] = data['eigenvalue_ratio'].diff(5)
48    data['vol_diff5'] = data['market_realized_vol_20d'].diff(5)
49    data['dispersion_diff5'] = data['market_cross_sectional_dispersion'].diff(5)
50    data['eigenvalue_ratio_slope10'] = rolling_slope(data['eigenvalue_ratio'], 10)
51    data['vol_slope10'] = rolling_slope(data['market_realized_vol_20d'], 10)
52    data['eigenvalue_ratio_slope20'] = rolling_slope(data['eigenvalue_ratio'], 20)
53    data['vol_slope20'] = rolling_slope(data['market_realized_vol_20d'], 20)
54    data['vol_of_vol_10d'] = data['market_realized_vol_20d'].rolling(10).std()
55    data['market_return_skew_20d'] = data['market_return'].rolling(20).skew()
56    data['vol_dispersion_corr_20d'] = data['market_realized_vol_20d'].rolling(20).corr(data['market_cross_sectional_dispersion'])
57
58    data['return_autocorr_20d'] = data['market_return'].rolling(20).apply(lambda x:
59    pd.Series(x).autocorr(lags=1), raw=False)
60    data['vol_zscore_60d'] = (data['market_realized_vol_20d'] -
61    data['market_realized_vol_20d'].rolling(60).mean()) /
62    data['market_realized_vol_20d'].rolling(60).std()
63    data['dispersion_zscore_60d'] = (data['market_cross_sectional_dispersion'] -
64    data['market_cross_sectional_dispersion'].rolling(60).mean()) /
65    data['market_cross_sectional_dispersion'].rolling(60).std()
66
67    cum_return = data['market_return'].cumsum()
68    data['drawdown_20d'] = cum_return - cum_return.rolling(20).max()
69    data['downside_vol_20d'] = data['market_return'].clip(upper=0).rolling(20).std()
70
71    data = data.dropna()
72
73    # 6. Return only the latest available row
74    return data.iloc[-1:]
75
76 tickers = universe
77 latest_row = latest_feature_extraction(tickers)
78 print(latest_row)
79
80 latest_features=latest_row[feature_cols]
```

Figure 17: Complete forecast latest-feature extraction code.

6.8 Regime-Driven Portfolio Decision

The final portfolio decision is also different only in the time interpretation.

6.8.1 Nowcast regime-to-portfolio mapping

The predicted current regime determines whether the eigenvector exposure constraint is active. The optimization is then solved and the portfolio weights and variance are reported.

```
Nowcast regime-driven portfolio

1  # Get the most recent row's features (or any specific date you want to check)
2  #latest_features = final_data[feature_cols].iloc[[-1]] # double brackets keep it as a DataFrame
3
4  regime_today = model.predict(latest_features)[0]
5
6  print("Predicted regime:", regime_today)
7
8  # Same constraint logic as before, now driven by the real prediction
9  if regime_today in ['transitioning', 'stressed']:
10     constraints3 = [
11         cp.sum(w3) == 1,
12         w3 >= 0,
13         cp.abs(exposure3) <= 0.15
14     ]
15 else:
16     constraints3 = [
17         cp.sum(w3) == 1,
18         w3 >= 0
19         # no exposure cap when regime is calm
20     ]
21
22 problem3 = cp.Problem(cp.Minimize(portfolio_variance3), constraints3)
23 problem3.solve()
24
25 problem3.solve()
26 print("Portfolio weights:", w3.value)
27 print("Portfolio variance:", problem3.value)
```

Figure 18: Complete nowcast regime-driven portfolio code.

The forecast model uses the same optimization mechanism but predicts the regime for tomorrow. No code after the portfolio-weight and portfolio-variance output is included in this model section, as requested; the later R^2 comparison is reported only in the Results section.

6.8.2 Forecast regime-to-portfolio mapping

The forecast model uses the same optimization mechanism but predicts the regime for tomorrow. No code after the portfolio-weight and portfolio-variance output is included in this model section; the later R^2 comparison is reported only in the Results section. The forecast prediction is explicitly labelled as the regime for tomorrow, and the predicted regime determines whether the exposure constraint is active.

```
Forecast regime-driven portfolio

1  # Get the most recent row's features (or any specific date you want to check)
2  #latest_features = final_data[feature_cols].iloc[[-1]] # double brackets keep it as a DataFrame
3
4
5  regime_tomorrow = model.predict(latest_features)[0]
6  print("Predicted regime for TOMORROW:", regime_tomorrow)
7
8  # Same constraint logic as before, now driven by the real prediction
9  if regime_tomorrow in ['transitioning', 'stressed']:
10     constraints3 = [
11         cp.sum(w3) == 1,
12         w3 >= 0,
13         cp.abs(exposure3) <= 0.15
14     ]
15 else:
16     constraints3 = [
17         cp.sum(w3) == 1,
18         w3 >= 0
19     ] # no exposure cap when regime is calm
20
21
22 problem3 = cp.Problem(cp.Minimize(portfolio_variance3), constraints3)
23 problem3.solve()
24
25 problem3.solve()
26 print("Portfolio weights:", w3.value)
27 print("Portfolio variance:", problem3.value)
```

Figure 19: Complete forecast regime-driven portfolio code.

7 Results

7.1 Classification Results

The supplied project evaluation reports approximately 90% cross-validated accuracy for both tasks:

Table 1: Classification results

Model	Task	Cross-validated accuracy
Nowcast	Same-day regime	$\approx 90\%$
Forecast	Next-day regime	$\approx 90\%$

The project report also records representative nowcast test-set precision, recall, and F1 ranges for calm, transitioning, and stressed classes.

7.2 Portfolio Exposure and Variance Results

The supplied notebook outputs show the intended effect of the eigenvector constraint: exposure is pushed from approximately -0.396 or -0.398 toward the cap of -0.15 , while the optimized variance increases.

Table 2: Portfolio optimization output from the supplied runs

Quantity	Run 1	Run 2
Baseline exposure	-0.3961673	-0.3981418
Constrained exposure	-0.1500000	-0.1500000
Baseline variance	9.6236755×10^{-5}	$9.80071399 \times 10^{-5}$
Constrained variance	$1.13968198 \times 10^{-4}$	$1.16212662 \times 10^{-4}$

Table 3: Run 1 stock-weight changes

Stock	Baseline	Constrained	Change
AAPL	0.065	0.027	Large decrease
JPM	0.055	0.160	Large increase
JNJ	0.446	0.329	Decrease
XOM	0.112	0.010	Large decrease
WMT	0.285	0.242	Decrease
CAT	0.036	0.232	Large increase

```

Exposure (baseline):      -0.3961673332602027
Exposure (constrained):   -0.150000000000000002
Variance (baseline):      9.623675532219908e-05
Variance (constrained):   0.000113968198548766

Stock  Baseline  Constrained  Change
AAPL   0.065     0.027  ↓↓ (big drop)
JPM    0.055     0.160  ↑↑ (big jump)
JNJ    0.446     0.329  ↓
XOM    0.112     0.010  ↓↓ (big drop)
WMT    0.285     0.242  ↓
CAT    0.036     0.232  ↑↑ (big jump)

print('nowcast for 4th september')

nowcast for 4th september

```

Figure 20: Supplied notebook output showing eigenvector exposure, baseline and constrained variance, and stock-level weight changes.

```

Exposure (baseline):      -0.39814183025929545
Exposure (constrained):   -0.14999999999999997
Variance (baseline):      9.800713988520978e-05
Variance (constrained):   0.000116212662119134

Stock  Baseline  Constrained  Change
AAPL   0.064     0.026  ↓↓ (big drop)
JPM    0.053     0.159  ↑↑ (big jump)
JNJ    0.447     0.330           ↓
XOM    0.113     0.010  ↓↓ (big drop)
WMT    0.287     0.243           ↓
CAT    0.036     0.233  ↑↑ (big jump)

print('forecast for 5th september')

forecast for 5th september

```

Figure 21: Supplied notebook output for the corresponding portfolio comparison run.

7.3 September 3 Nowcast Versus Forecast Result

The final supplied observations are particularly important because they compare the two pipelines on the same information date.

Forecast for September 3 based on September 2 features:

Predicted regime for TOMORROW: transitioning

Nowcast for September 3:

Predicted regime: transitioning

Thus, both pipelines produced the same regime class for September 3.

7.4 September 3 Portfolio Weights

The nowcast portfolio produced:

$$w_{\text{nowcast}} = \begin{bmatrix} 0.02691662 & 0.15962744 & 0.32908587 & 0.00976785 & 0.24215061 & 0.23245160 \end{bmatrix}.$$

Its portfolio variance was

$$V_{\text{nowcast}} = 0.00011396819854876596.$$

The forecast portfolio produced:

$$w_{\text{forecast}} = \begin{bmatrix} 0.02590968 & 0.15896371 & 0.32977993 & 0.00950186 & 0.24275456 & 0.23309025 \end{bmatrix}.$$

Its portfolio variance was

$$V_{\text{forecast}} = 0.00011621266211913426.$$

7.5 Similarity of the Two September 3 Portfolios

The reported R^2 between the nowcast and forecast portfolio-weight vectors is

$$R^2 = 0.9999654945268012.$$

This is extremely close to one and therefore indicates that the two six-dimensional weight vectors are almost identical in the reported run. It should be interpreted as a similarity measure between the two portfolio outputs, not as a measure of investment performance or predictive accuracy.

8 Benchmarks

8.1 Single Eigenvalue-Ratio Threshold Baseline

A single-signal regime rule based on the eigenvalue ratio was implemented as a benchmark. The supplied report states that it agreed with the ML-derived regime assignment in approximately 51% of test observations.

For a three-class problem, random assignment has an expected agreement of approximately

$$\frac{1}{3} \approx 33.3\%.$$

The comparison therefore indicates that the eigenvalue ratio contains useful information but is insufficient by itself to reproduce the multi-feature regime assignment reliably.

8.2 Unconstrained Minimum-Variance Portfolio

The unconstrained long-only minimum-variance portfolio is the portfolio benchmark against which the regime-aware constrained portfolio is compared. The regime-aware overlay is not

expected to minimize variance as aggressively because it deliberately restricts exposure to the dominant systemic direction.

9 Experiments

9.1 Feature-Set Size Experiment

The project compared 8, 15, and 19 engineered features. The supplied report states that the compact eight-feature specification performed better than the larger sets, which showed signs of overfitting: training accuracy increased while test accuracy degraded.

The final eight-feature set therefore represents an empirical feature-selection result rather than an assumption that more engineered variables must improve performance.

9.2 Nowcast Versus Forecast Experiment

The two models answer different questions:

$$\text{Nowcast: } X_t \rightarrow \text{Regime}_t,$$

$$\text{Forecast: } X_t \rightarrow \text{Regime}_{t+1}.$$

The September 3 experiment shows that both pipelines produced the transitioning regime, and their portfolio weights were almost identical under the reported R^2 comparison.

9.3 Eigenvector Constraint Experiment

The portfolio experiment compares

$$w_{\text{baseline}}$$

with

$$w_{\text{constrained}}.$$

The constraint moves the exposure from approximately -0.396 to -0.15 , demonstrating that the optimization is actively controlling the dominant eigenmode rather than merely changing weights incidentally.

10 Ablation and Failure-Mode Analysis

10.1 VIX Label Leakage

An initial implementation generated labels directly from VIX while also including VIX as a classifier feature. This produced an implausible approximately 99.7% accuracy. The project correctly identified this as label leakage.

10.2 Universe-Index Mismatch

VIX is derived from a broad S&P 500 options market, while the project operates on a six-stock universe. Using VIX as the direct ground truth for the smaller universe can therefore create a mismatch between the target and the actual dependence structure being modelled.

10.3 Removal of Direct VIX From the Classifier

The final classifier does not use the VIX proxy as an input feature. It is used to construct the regime target, after which the proxy is removed from the feature vector. This is a critical control against a subtler version of the same leakage problem.

10.4 Feature-Set Ablation

The reduction from the larger candidate feature set to eight features is an important ablation. The project found that the smaller set generalized better, suggesting that some of the additional variables were redundant or contributed to overfitting.

11 Discussion

11.1 What the Model Actually Learns

The classifier does not simply learn a single volatility threshold. It receives information about:

- the level of realized market volatility;
- cross-sectional dispersion;
- the leading and second eigenvalues;
- the dominance ratio of the leading eigenvalue;
- recent changes in the eigenvalue ratio;
- recent changes in volatility;
- recent changes in dispersion.

The model therefore combines level, cross-sectional structure, and short-horizon dynamics.

11.2 Why the Portfolio Overlay Is Economically Meaningful

The leading eigenvector provides a direction in the six-dimensional asset space. Constraining $w^\top v_1$ means the optimizer is explicitly prevented from constructing a portfolio that is too aligned with that dominant direction during adverse regimes.

This is different from simply reducing total variance. The unconstrained optimizer may prefer a low-variance solution that nevertheless has substantial exposure to the dominant market mode. The regime-aware optimizer adds an explicit systemic-risk dimension.

11.3 Interpretation of the September 3 Result

Both models classified September 3 as transitioning. Their portfolio weights were also very close, with $R^2 = 0.9999654945268012$. This indicates strong agreement between the same-day and one-step-ahead pipelines for this particular prediction date.

The forecast portfolio variance is slightly higher than the nowcast variance in the reported numbers:

$$0.00011621266211913426 > 0.00011396819854876596.$$

This difference does not indicate that the forecast model is worse. The two optimization runs are based on slightly different feature-generation paths and the variance is a consequence of the resulting optimization solution.

11.4 Backtesting Limitation

The attempted walk-forward backtest exposed a methodological instability. Under one threshold calibration, the rule-based strategy activated its protective constraint in fewer than 20% of test observations and behaved similarly to the unconstrained baseline. Under another calibration, the constraint activated in more than 80% of observations and the strategy appeared to outperform substantially.

Because an implementation choice could therefore change the apparent performance conclusion, the project does not claim robust comparative backtest performance. Instead, this instability is treated as a finding that motivates more rigorous future testing.

12 Future Improvements

1. The model has currently been tested on only 6-stock and 10-stock universes.

2. The R^2 comparison between nowcast and forecast is a similarity measure of the predicted and actual regimes and of the corresponding portfolio weights; it is not a return statistic.
3. The model should be tested on a broader universe containing at least 50 stocks to evaluate scalability and robustness.
4. At present, the nowcast and forecast models are implemented as two separate files. In future, they will be integrated into a single unified tool with deployment capability.

13 Conclusion

The project develops a complete regime-aware portfolio architecture in which market dependence is measured through correlation eigenstructure, regimes are inferred using machine learning, and the resulting regime signal changes the feasible portfolio set.

The most important empirical result is that both the nowcast and forecast classifiers achieve approximately 90% cross-validated accuracy on the project’s constructed three-class regime target. The leading eigenvalue/eigenvector structure also behaves consistently with the economic interpretation used by the framework.

The portfolio experiment demonstrates that the eigenvector exposure constraint is mechanically effective: it moves exposure toward the prescribed ± 0.15 limit and redistributes portfolio weights away from assets contributing strongly to the dominant eigenmode.

For September 3, both the nowcast and forecast models classify the regime as transitioning. Their portfolio weights are extremely similar, with

$$R^2 = 0.9999654945268012.$$

The project therefore provides evidence that the proposed architecture can connect regime detection to portfolio construction in a transparent and mathematically controlled manner. It does not, however, claim robust investment outperformance because the walk-forward backtest was found to be sensitive to threshold calibration.

14 Project Repository and Reproducibility

The complete implementation of the proposed regime-aware portfolio optimization framework, including the data-processing pipeline, machine-learning models, regime classification procedures, eigenstructure analysis, and portfolio optimization code, is available in the project’s GitHub repository.

[https://github.com/k-means-kriss/
Regime-Aware-Portfolio-Optimization-Using-
Correlation-Eigenstructure-and-Machine-
Learning-Based-Mark](https://github.com/k-means-kriss/Regime-Aware-Portfolio-Optimization-Using-Correlation-Eigenstructure-and-Machine-Learning-Based-Mark)

The repository contains the notebooks and supporting implementation required to reproduce the main experiments and results reported in this project. The two principal notebooks correspond to the nowcast and forecast implementations of the proposed framework. The implementation sections of this report reproduce the supplied notebook code through the portfolio-weight and portfolio-variance outputs; the later notebook cells used for the R^2 comparison are intentionally not reproduced in the main model section, with the resulting R^2 value reported in the Results section.

15 References

Girnyk, M. A. (2026). *Correlation Structures and Regime Shifts in Nordic Stock Markets*. Applied Mathematical Finance, 33(1), 46–71.
<https://arxiv.org/abs/2601.06090>.